

## **CSE 331/531 Project 2:**

### **32-bit ALU Design for MIPS Architecture**

**Deadline: 30/11/2024 Second Project Exam**

#### **Objective:**

In this project, students will design a 32-bit ALU (Arithmetic Logic Unit) to integrate into a MIPS processor architecture. The ALU will be implemented in Structural Verilog and should support all integer operations required by MIPS, excluding floating-point operations. This ALU will be modular, using submodules for each operation and integrating them into a top-level module. Students will test functionality through simulation, without needing to submit a report.

#### **Requirements:**

##### **1. Design Structure:**

- Implement a 32-bit ALU that supports MIPS integer operations, excluding floating-point functions.
- The ALU should use Structural Verilog and be composed of multiple submodules, each responsible for specific operations.
- Minimize hardware usage by consolidating related operations (addition/subtraction in one module and left/right shift in another).

##### **2. ALU Submodules and Descriptions:**

Each submodule performs a specific function within the ALU. Below are the required submodules, their functions, inputs, and outputs:

- Arithmetic Module (**arithmetic\_unit**)
  - Purpose: Performs both addition and subtraction.
  - Inputs:
    - A (32-bit), B (32-bit): Operands.
    - Sub (1-bit): Control signal, where 0 indicates addition and 1 indicates subtraction.
  - Outputs:
    - Result (32-bit): Result of the addition or subtraction.
    - CarryOut (1-bit): Carry-out flag for addition or borrow flag for subtraction.
  - Functionality: Uses the Sub signal to determine whether to add or subtract B from A. When Sub = 1, it inverts B and adds 1 (two's complement) to achieve subtraction.
- Bitwise Operation Modules:

- **AND Module (and\_gate)**
  - Purpose: Performs bitwise AND operation.
  - Inputs: A (32-bit), B (32-bit)
  - Output: Result (32-bit)
- **OR Module (or\_gate)**
  - Purpose: Performs bitwise OR operation.
  - Inputs: A (32-bit), B (32-bit)
  - Output: Result (32-bit)
- **XOR Module (xor\_gate)**
  - Purpose: Performs bitwise XOR operation.
  - Inputs: A (32-bit), B (32-bit)
  - Output: Result (32-bit)
- **NOR Module (nor\_gate)**
  - Purpose: Performs bitwise NOR operation.
  - Inputs: A (32-bit), B (32-bit)
  - Output: Result (32-bit)
- **Shift Module (shift\_unit)**
  - Purpose: Performs both logical left and logical right shifts.
  - Inputs:
    - A (32-bit): Data to be shifted.
    - ShiftAmount (5-bit): Number of positions to shift.
    - Dir (1-bit): Direction of shift; 0 for left shift and 1 for right shift.
  - Output: Result (32-bit): The shifted output.
  - Functionality: Uses the Dir signal to determine the shift direction. When Dir = 0, it shifts A left; when Dir = 1, it shifts A right.
- **Set on Less Than Module (slt)**
  - Purpose: Compares A and B and outputs 1 if  $A < B$ , otherwise 0.
  - Inputs: A (32-bit), B (32-bit)
  - Output: Result (1-bit)
- **Zero Detector Module (zero\_detector)**
  - Purpose: Checks if the ALU result is zero (used for branch instructions).

- Input: Result (32-bit): Output of the ALU operation.
- Output: Zero (1-bit): Flag that is 1 if Result is zero, otherwise 0.

### 3. Top-Level ALU Module (alu\_top):

- Purpose: The main ALU module that integrates all submodules. It takes control signals, selects the appropriate submodule, and produces the final ALU output.
- Inputs:
  - A (32-bit), B (32-bit): Operands for the ALU.
  - OpCode (4-bit): Operation code that determines which operation the ALU performs (e.g., 0000 for add, 0001 for subtract).
- Outputs:
  - Result (32-bit): Output of the chosen ALU operation.
  - Zero (1-bit): Flag indicating if Result is zero (for branch instructions).
  - CarryOut (1-bit): Flag for carry-out or overflow, relevant for arithmetic operations.

### 4. Simulation and Testing:

- Create a testbench to verify each ALU operation with a variety of test cases.
- Suggested Test Vectors:
  - Addition and Subtraction: Use boundary cases (e.g., 0xFFFFFFFF, 0x00000000), overflow tests, and typical positive/negative values.
  - Bitwise Operations: Test with alternating bits (0xAAAAAAAA, 0x55555555), all bits set (0xFFFFFFFF), and all bits cleared (0x00000000).
  - Shift Operations: Test shifts of 0, 1, and 31 positions in both directions with various inputs.
  - SLT Operation: Use cases where A is greater than, less than, and equal to B, with both positive and negative values.
  - Zero Detection: Confirm detection accuracy when the Result is zero.

This design minimizes hardware usage by reusing modules for related operations (addition/subtraction, shift left/right) and aligns with structural Verilog design practices for efficient ALU functionality.